



Digital Integrated Circuit Design

Lecture-9: Fixed-Point Analysis

Chih-Feng Wu (吳志峰), PhD.

*Department of Electronics Engineering, Chang Gung University,
Taoyuan, Taiwan (ROC)*

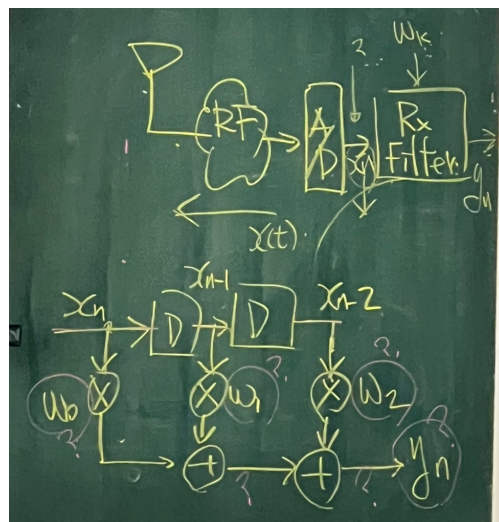
May 25, 2026

Chih-Feng Wu



Outline

- ❑ Fixed-Point Analysis
- ❑ Demo for Fixed-Point Analysis in MATLAB

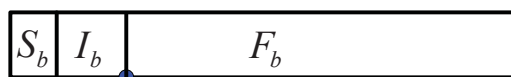


Chih-Feng Wu

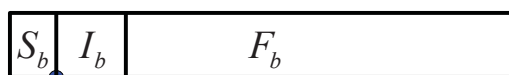
Fixed-Point Consideration

□ Finite-wordlength implementation

□ Format of the finite wordlength



left shift I_b -bit



■ $\langle S_b, I_b, F_b \rangle$

■ S_b : the number of sign bits; In general, $S_b=1$ or 0;

• $S_b=0$: the positive number

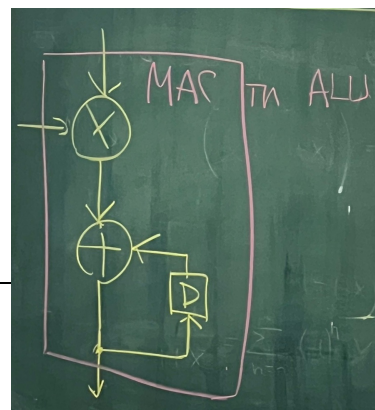
• $S_b=1$: the negative number

■ I_b : the no. of integer bits

■ F_b : the no. of fractional bits

■ •: the fractional (decimal) point

Chih-Feng Wu



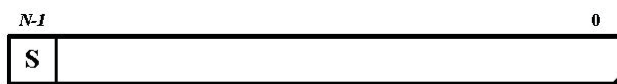
3

Fixed-Point Number Representation

□ Finite-wordlength representation

■ N -bit fixed-point using 2's complement number

$$x = -b_{N-1}2^{N-1} + b_{N-2}2^{N-2} + \dots + b_1 2^1 + b_0 2^0$$

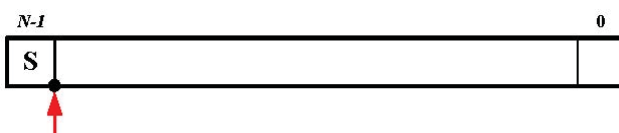


A binary decimal point
Radix point
Binary point

■ Difficult to work due to possible overflow and scaling problems

■ Normalize this number to be binary fractional representation

$$x' = -b_{N-1}2^0 + b_{N-2}2^{-1} + \dots + b_1 2^{-(N-2)} + b_0 2^{-(N-1)}$$



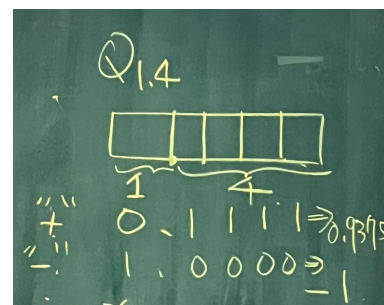
Chih-Feng Wu

4

Fixed-Point Number Representation (cont'd)

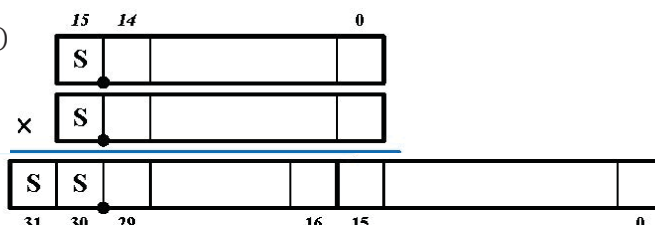
Q-number format (Q-format representation) $Q_{i,f}$ (or $Q_{i,f}$)

- i : the number of **integer** bits including a **sign-bit**
- f : the number of fractional bits
- $N=16$, 15 fractional bits: $Q_{1.15}$
- $Q_{1.4}$: max. = 0.9375 and min. = -1.0



Operation rules for Q-number format

- $Q_{i,m} + Q_{i,m} \Rightarrow Q_{i+1,m}$
- $Q_{i,m} + Q_{i,n} \Rightarrow Q_{i+1,m} (m > n)$
- $Q_{i,m} \times Q_{i,n} \Rightarrow Q_{i,m+n}$



For example

- $Q_{1.15} \times Q_{1.15} \Rightarrow Q_{1.30}$

sign bit

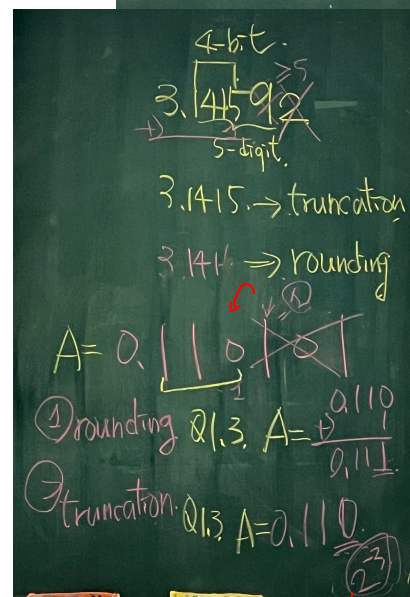
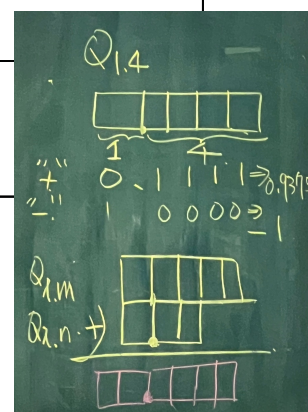
Chih-Feng Wu

Fixed-Point Arithmetic

- 無條件捨去 四捨五入 Truncation (rounding) error
 - Reduce the wordlength after multiplication
- Propagation error (Quantized error)
 - Error term caused by the arithmetic operations
- Clipping (saturation) error

Signal-to-quantization-noise ratio (SQNR)

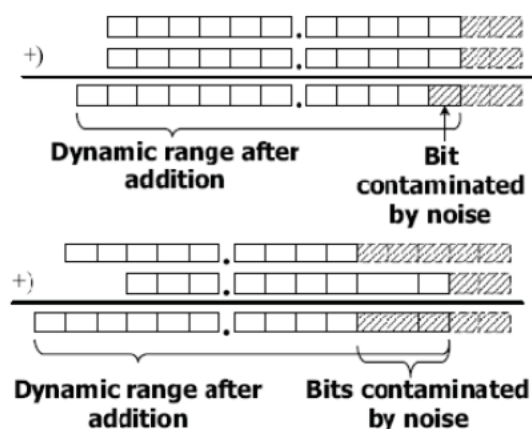
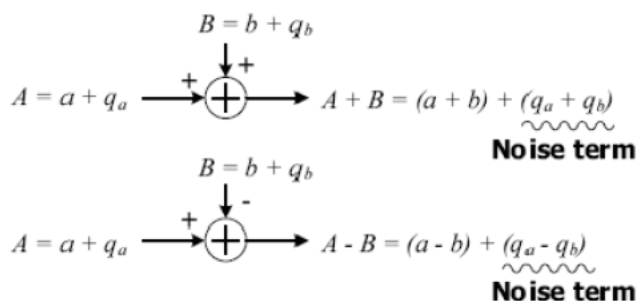
- $SQNR = 10 \cdot \log_{10} \frac{\sigma_s^2}{\sigma_q^2}$
- $\sigma_s^2 = E[|S|^2]$
- $\sigma_q^2 = E[|Q_e|^2]$ and $Q_e = \text{Ideal} - \text{Fixed-point} = S - S_q$
- Q_e : the quantized error (noise)



Quantized Error

Chih-Feng Wu

Propagation Error for Addition/Subtraction



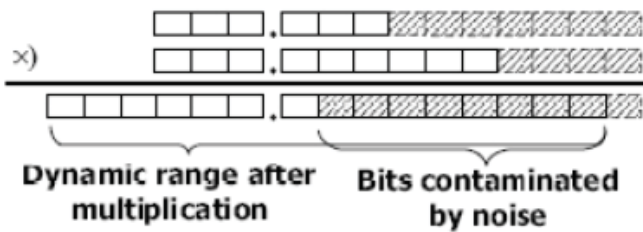
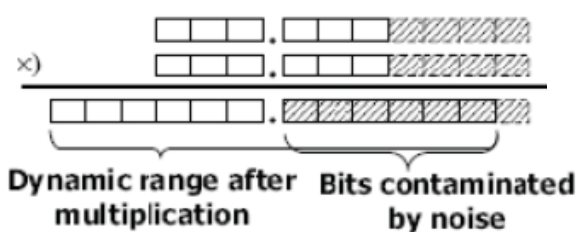
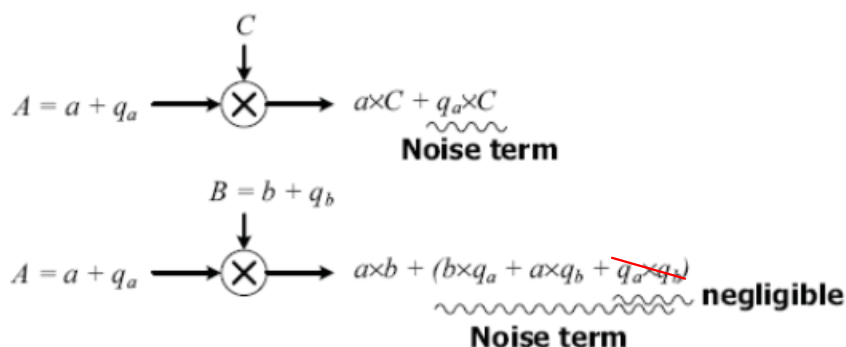
$$SQNR = 10 \cdot \log_{10} \frac{(a \pm b)^2}{(q_a \pm q_b)^2} = 20 \cdot \log_{10} \frac{a \pm b}{q_a \pm q_b}$$

$$\text{Quantized-Power} = 10 \cdot \log_{10} \frac{1}{(q_a \pm q_b)^2} = 20 \cdot \log_{10} \frac{1}{q_a \pm q_b}$$

Chih-Feng Wu

7

Propagation Error for Multiplication



$$SQNR = 10 \cdot \log_{10} \frac{(a \times b)^2}{(b \times q_a + a \times q_b + \cancel{q_a \times q_b})^2}$$

Chih-Feng Wu

8

Overflow Discussion for Q-Number Format

□ Addition for Q-number format

■ $a + b, a = -1.25 \Rightarrow 10.11(Q_{2.2}); b = 3.25; \Rightarrow 011.010(Q_{3.3})$

■ $Q_{2.2} + Q_{3.3} \Rightarrow Q_{3.2} + Q_{3.3} = Q_{3.3}$ (without overflow)

$\Rightarrow Q_{3.3} + Q_{3.3} = Q_{3.3}$ (without overflow)

$\Rightarrow a + b = 2 \Rightarrow \mathbf{1}010.000$

$\Rightarrow _010.000$ (discard “1” $\Rightarrow Q_{3.3}$)

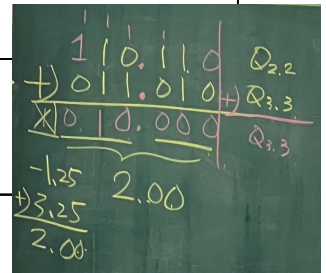
□ $Q_{ia.fa} + Q_{ib.fb} = Q_{ic.fc}$

■ $ic = \max(ia+1, ib+1)$ and $fc = \max(fa, fb)$

■ It is possible to occur overflow when the summation of two N-bit number can lead to (N+1)-bit number.

■ We have to sure that the overflow has not occurred.

Chih-Feng Wu



Overflow Discussion for Q-Number Format (cont'd)

□ Calculate the following $a + b$

■ $a = -1.25 \Rightarrow 10.11(Q_{2.2}); b = -3.875; \Rightarrow 100.001(Q_{3.3})$

■ $Q_{2.2} + Q_{3.3} \Rightarrow a + b = -5.125 \Rightarrow \mathbf{1}010.111(Q_{4.3})$

■ Use $Q_{4.3}$ as the output format to avoid overflow when addition two $Q_{3.3}$ numbers

■ $a + b = 1110.11 + 1100.001 = \mathbf{1}1010.111$ (discard “1”)
 $\Rightarrow _1010.111(-5.125)$

■ Avoiding overflow using **guard** bits

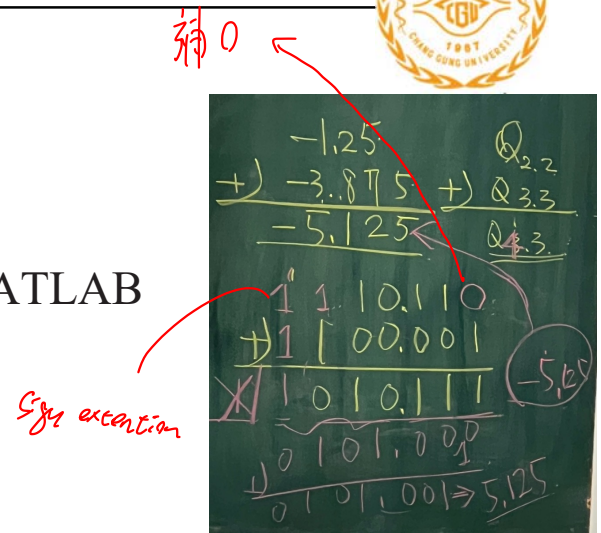
- The guard bits are generated by the signed-extension.

Chih-Feng Wu



Outline

- ❑ Fixed-Point Analysis
- ❑ Demo for Fixed-Point Analysis in MATLAB



Fixed-Point Simulation in MATLAB

- ❑ Toolbox for fixed-point simulation

■ `var = sfi(v,w,f);`

- `v`: input value (or **entire** array)
- `w`: the word-length of the fixed-point
- `f`: the number of fractional bits

■ `fprintf(fid,'%s \n', hex(var));`

■ `fprintf(fid,'%s \n', bin(var));`

■ `var=trunc(v, 't', t_bit);`

- `v`: input value
- `'t'`: truncation
- `t_bit`: the fractional bit after truncation

rounding $\hat{x} + 1 \Rightarrow$ adder

Fixed-Point Simulation in MATLAB (cont'd)

- Example: Fixed-point representation for π using $\langle 1, 2, 5 \rangle$

- `trunc(pi, 't', 6)`

- `sfi(pi, 9, 5)`

```
>> Qx = sfi(pi, 9, 6)
Qx =
    3.140625000000000
```

```
DataTypeMode: Fixed-point: binary point scaling
Signedness: Signed
WordLength: 9
FractionLength: 6
```

```
>> Qx.hex
ans =
    '0c9'
>> Qx.bin
ans =
    '011001001'
```

$$\pi = 3.14159 \approx 653$$

$$\langle 1, 2, 6 \rangle = Q_{36}$$

$$\frac{\text{floor}(\pi \times 2^6)}{2^6} = \frac{201}{2^6}$$

$$= 3.140625$$

`trunc(pi, 't', 6)`

```
>> Q = floor(pi*2^6)/2^6
Q =
    3.140625000000000
>> q = quantizer([9,6])
q =

    DataMode = fixed
    RoundMode = floor
    OverflowMode = saturate
    Format = [9 6]
>> Qx = num2bin(q,Q)
Qx =
    '011001001'
```

- `fprintf(fid, '%s \n', hex(Qx));`

Chih-Feng Wu

13

$$\pi = 3.14159 \approx 653$$

$$\langle 1, 2, 6 \rangle = Q_{36}$$

$$\frac{\text{floor}(\pi \times 2^6)}{2^6} = \frac{201}{2^6}$$

$$= 3.140625$$

$$\frac{011001001}{3} = 6 \text{ bits}$$

$$\frac{1}{5} = 10 \times \log_{10}(\pi - 3.140625) \approx -62.3 \text{ dB}$$

$$\text{trunc}(x, f);$$

$$\frac{\text{floor}(x \times 2^f)}{2^f}$$

floating # of fraction bits

fixed-point NO.

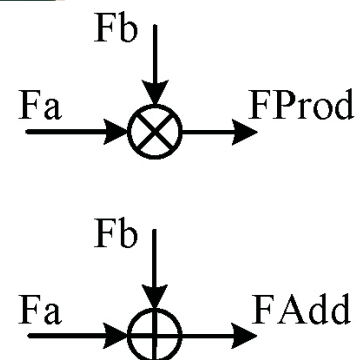
Demo

□ Multiplication and addition

- Fa and Fb: $\langle 1, 2, 5 \rangle$ or Q3.5
- FProd: $\langle 1, 4, 10 \rangle$ or Q5.10
- FAdd: $\langle 1, 3, 5 \rangle$ or Q4.5

■ Input data (FLT)

```
Fa = [-2.1383552694, -0.8395887473, 1.3545943280, -1.0721552884, 0.9609538697];
Fb = [0.1240498000, 1.4366966227, -1.9608999994, -0.1976982260, -1.2078454853];
```



$$\begin{aligned} &\text{+) } \langle 1, 2, 5 \rangle \\ &\text{+) } \langle 1, 2, 5 \rangle \\ &\text{+) } \langle 1, 3, 5 \rangle \\ &\text{+) } \langle 1, 2, 5 \rangle \\ &\text{+) } \langle 1, 2, 5 \rangle \\ &\text{+) } \langle 1, 4, 10 \rangle \\ &\text{+) } \langle 1, 2, 5 \rangle \\ &\text{+) } \langle 1, 0, 10 \rangle \\ &\text{+) } \langle 1, 2, 15 \rangle \end{aligned}$$

Chih-Feng Wu

14

Demo (cont'd)

□ Program demo

```

37 for n=1:1:N
38     % FLT
39     FP(n) = Fa(n)*Fb(n);
40     FA(n) = Fa(n) + Fb(n);
41     fprintf(fid1x,'%13.10f %13.10f %13.10f %13.10f\n', ...
42             FP(n), FA(n), Fa(n), Fb(n));
43
44     % FXP
45     FaX = Fa(n);
46     FbX = Fb(n);
47     FaXP= trunc(FaX, 't', 5);
48     FbXP= trunc(FbX, 't', 5);
49     FaXP=sfi(FaXP, 8, 5);
50     FbXP=sfi(FbXP, 8, 5);
51     FProdXP= sfi(FaXP * FbXP, 15, 10);
52     FAddXP= sfi(FaXP + FbXP, 9, 5);
53
54     fprintf(fid2x,'%s (%19.10f) %s (%9.5f) %s (%8.5f) %s (%8.5f)\n', ...
55             hex(FProdXP), FProdXP, hex(FAddXP), FAddXP, hex(FaXP), FaXP, hex(FbXP), FbXP);
56 end

```

Chih-Feng Wu

15

Demo (cont'd)

□ Input/output data (FXP)

1	7f31 (	-0.2021484375)	1be (	-2.06250)	bb (	-2.15625)	03 (	0.09375)
2	7b41 (	-1.1865234375)	012 (	0.56250)	e5 (	-0.84375)	2d (	1.40625)
3	756b (	-2.6455078125)	1ec (	-0.62500)	2b (	1.34375)	c1 (	-1.96875)
4	00f5 (	0.2392578125)	1d6 (	-1.31250)	dd (	-1.09375)	f9 (	-0.21875)
5	7b6e (	-1.1425781250)	1f7 (	-0.28125)	1e (	0.93750)	d9 (	-1.21875)

FProd **FAdd** **Fa** **Fb**
 <1, 4, 10> <1, 3, 5> <1, 2, 5>

□ Input/output data (FLT)

1	-0.2652625435	-2.0143054694	-2.1383552694	0.1240498000
2	-1.2062343177	0.5971078754	-0.8395887473	1.4366966227
3	-2.6562240170	-0.6063056714	1.3545943280	-1.9608999994
4	0.2119631985	-1.2698535144	-1.0721552884	-0.1976982260
5	-1.1606837931	-0.2468916156	0.9609538697	-1.2078454853

FProd **FAdd** **Fa** **Fb**

```

Fa = [-2.1383552694, -0.8395887473, 1.3545943280, -1.0721552884, 0.9609538697];
Fb = [0.1240498000, 1.4366966227, -1.9608999994, -0.1976982260, -1.2078454853];

```

Chih-Feng Wu

16

